INFO 6150 / Prof. Fernando Augusto López Plascencia / Northeastern University

Photo by Eren Li: https://www.pexels.com/photo/photo-of-person-using-braille-7188797/

Week 12

Usability and Accessibility

In this lesson:

- Usability principles
- Usability best practices
- What is Accessibility
- Select accessibility best practices

Usability

Let’s go back to usability now

Now that we can create websites and apps, let’s review what web usability is and how to care for it.

The components of usability

At this point we'll say that usability is a **quality attribute of an interface.**

The Nielsen-Norman Group says usability of a website or app has 5 components:

- **Learnability**: How easy is it to accomplish tasks in it, for the first time?
- **Efficiency**: After the first time, how quickly can users perform those tasks again?
- **Memorability**: After not using it for a while, how fast can users become proficient again?
- **Errors**: How many errors do the users make, how bad are they, and how easily can they recover from them?
- **Satisfaction**: How pleasant to use is the design?

Usability vs. utility

Utility is also important. It means: whether the website or app actually does what the user needs.

Usability means: how easy and pleasant it is to use.

Usefulness = usability + utility.

Usability is important because...

If the users cannot do what they came to the website or app to do, they get **frustrated.**

- If this is a public website or app, they will simply leave.
- If this is an internal website or app, this means productivity lost.

People get frustrated when:

- They can’t find the options they’re looking for.
- The information is hard to understand, or useless for their use cases.
- The website is difficult to use **for them.**

You’ve been already working on usability:

The techniques we’ve explored so far all aid in usability:

- Checking your competitor’s work allows you to discover not just what, but also **how** to do / not to do things
- UX studies for discovery can create more usable websites / apps
- Low and high definition prototypes also help.
    - Ask your potential users to look at them and get feedback. Here’s a case study.

Usability testing

Once the website or app is built*, you can start doing **usability testing sessions**.

Their goals:

- Identifying problems in the design
- Uncovering opportunities to improve
- Learning more about the user and how they behave

*You can also do this with high-fidelity mockups, up to a point.

Usability testing

This is **one of the more powerful techniques I've used in my career.** It's useful in products both new and old.

Two variants: **guided test** and **raw observation**.

In the "guided test":

- Sit the user in front of your website or app.
- Ask them to do specific things.
- **STAY SILENT.** Observe them do things.
    - If they struggle, resist the temptation to help them out.
    - If they make comments, write them down.
    - Note what they're thinking, how they're feeling.

This is useful for users at every experience level.

Usability testing

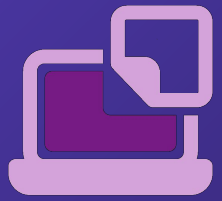
This is **one of the more powerful techniques I've used in my career.** It's useful in products both new and old.

Two variants: **guided test** and **raw observation**.

The "raw observation" is similar and more useful for already existing products.

You will just observe the user of a product they're already familiar with, as they go with their usual activities.

Since this test is unguided, it is useful to understand usage patterns that you might have been missing.



Created by HideMaru
from Noun Project

Try it out!

Do **guided usability testing** on the websites of two different airlines.

- Get the cost of a one-way ticket from San Francisco to New York, for November 1st, for one adult.

Which one was easier? What would you suggest to improve in each case?

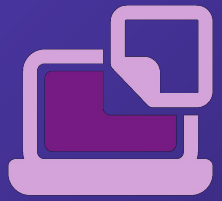
Icon by HideMaru for Noun Project

Usability best practices

- Organization and labeling
- Site hierarchy and navigation
- Page layout
- Visual design
- Images

Usability best practices: Organization and labeling

- Organization
  - Follow Information Architecture principles to organize the information and make it easily findable
- Labeling (“information scent”)
  - Be consistent with how things are named.
  - People scan titles first, and use them to decide if they will keep reading that section.
  - To decide on titles, consider how your **users** think, rather than your clients.
    - Follow the “don’t make me think” principle: obvious things are best
    - Avoid using technical words, except when the audience is technical

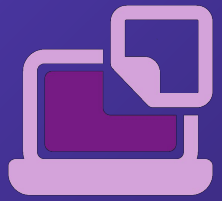


Created by HideMaru
from Noun Project

Usability best practices: Organization and labeling

What is a good title for a page or page section about:

- In an app like Google Maps, an ordered list of all locations corresponding to a search?



Created by HideMaru
from Noun Project

Usability best practices: Organization and labeling

What is a good title for a page or page section about:

- In a wiki such as Wikipedia:
  - the functionality to discuss the content of a page with other users?
  - the functionality to change the content of the page?

Icon by HideMaru for Noun Project

Usability best practices: Site hierarchy and navigation

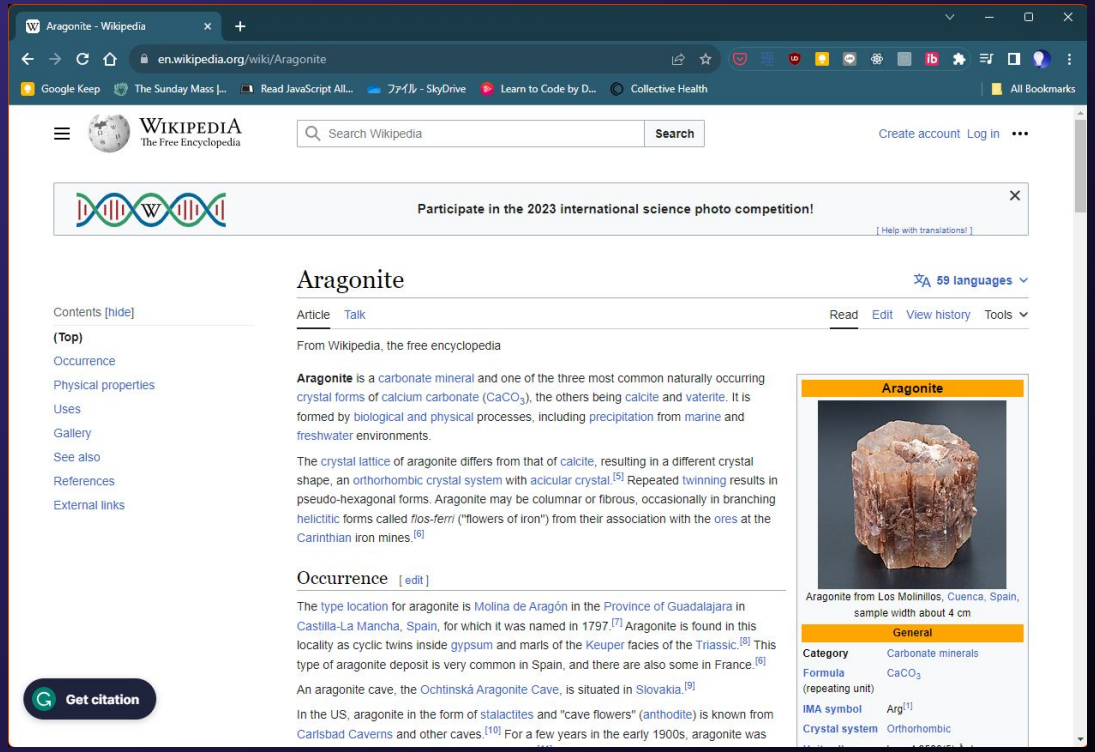
Site hierarchy:

- Create a sitemap page; users may prefer this over a search function

Menus and navigation:

- Tell users about the site's hierarchy
- Positioning:
  - Main menus **on top;** these reflect the most needed functions
  - Secondary navigation either on a sidebar, or as drop-downs
  - On mobile, can be **under a menu icon**
  - Additional links (eg. reflecting hierarchy) may be in the footer

Usability best practices: Site hierarchy and navigation



Usability best practices: Site hierarchy and navigation

Aragonite - Wikipedia

en.wikipedia.org/wiki/Aragonite

Google Keep | The Sunday Mass |... | Read JavaScript All... | ファイル - SkyDrive | Learn to Code by D... | Collective Health | All Bookmarks

Contents [hide]

(Top)
Occurrence
Physical properties
Uses
Gallery
See also
References
External links

18. ^ Mychaluk, Keith A.; Levinson, Alfred A.; Hall, Russel L. (Spring 2001). "Ammolite: Iridescent fossilized ammonite from southern Alberta, Canada" (PDF). *Gems & Gemology*. **37** (1): 4–25. doi:10.5741/GEMS.37.1.4. Retrieved 1 August 2021.
19. ^ Wang, Jang; Jahren, A. Hope; Amundsen, Ronald (1996). "Potential For [Carbon 14] Dating Of Biogenic Carbon In Hackberry (Celtis) Endocarps" (PDF). *Quaternary Research*. **47**: 337–343. doi:10.1006/qres.1997.1894. S2CID 49232599. [permanent dead link]

2010). "Magnetic water treatment for scale control in heating and alkaline conditions". *Desalination and Water Treatment*. **22** (1–3): 65–71. Bibcode:2010DWatT..22...65K. doi:10.5004/dwt.2010.1549.

External links [edit]

- The Ochtinska Aragonite Cave in Slovakia
- Kosovo Caves Aragonite Formations

Authority control databases	[hide]
National	Czech Republic
Other	Encyclopedia of Modern Ukraine

Categories: Calcium minerals | Carbonate minerals | Cave minerals | Aragonite group | Orthorhombic minerals | Minerals in space group 62 | Luminescent minerals | Evaporite | Minerals described in 1797

This page was last edited on 19 November 2023, at 22:05 (UTC).

Text is available under the Creative Commons Attribution-ShareAlike License 4.0; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.

Privacy policy | About Wikipedia | Disclaimers | Contact Wikipedia | Code of Conduct | Developers | Statistics | Cookie statement | Mobile view

Get citation

Usability best practices: Page Layout and Wireframing

Page Layout:

- People are used to elements in certain places already. Do not vary these too much for the best usability.
- Where do you expect the following to be in a webpage?
  - Logo or website name
  - Copyright information
  - “Search” functionality

Usability best practices: Page Layout and Wireframing

Wireframing:

- When creating the website or app, use them to agree with the stakeholders on:
  - Information in the pages
  - Form controls and other interactive elements
  - Placement and priority of those elements

Usability best practices: Visual Design

When creating the visual design, keep in mind:

- Create visible titles to guide the user
- Titles should be visible according to their hierarchy
  - Use font size, contrast, blank space…
- Less important parts of the page should have less visual weight

Usability best practices: Visual Design

Text should be legible!

- Use the default font sizes, not smaller, for main text
  - Usually `html { font-size: 16px; }`
  - Don’t go below 12px
- Tweak line heights (`line-height` between 1 and 1.5 is great)
- Ensure good contrast between text and background, including for hover text
- Lines should be around 80 characters long
  - Divide text in columns to improve this
- Don’t use display fonts for body text

Usability best practices: Images

Images and illustrations are a powerful vehicle for your content.

- Use **professional images** where possible
- Make sure the images will make sense for your audience. Consider culture; avoid biases.
    - If using people, ideally they should represent your audience
- Too many images can also be a problem. Ensure they do not compete against each other

Usability best practices: Images

Technical aspects:

- Do **not** stretch images. Make sure they fit the proportions for their designed space.
- Compress images, but make sure they are not pixelated
- Choose the right format: **lossy** vs **lossless**
  - Lossy: The file can be compressed easily, but the image will lose quality.
  - Lossless: The file may not compress as much but it will never lose quality.

Photo by Jenkin Shen:
https://www.pexels.com/photo/skyscraper-against-blue-sky-15774476/

Image formats (refresher from Week 2)

Common image formats for the web:

JPG	Lossy	Default for photos. Compressed. Can adjust compression % for a smaller file size
PNG	Lossless	Used for illustrations with transparency. Big file sizes.
GIF	Lossless	Traditional format for animations and transparency. Big files.
SVG	Lossless	Used for illustrations. Small file sizes. Vectorial: contains instructions to draw, rather than an image itself.
WebP	Can be either	New-ish format; better compression than JPG and PNG. Supports animations and transparency.
APNG	Lossless	New format. PNG with animation capabilities.
AVIF	Can be either	New format, supports both images and animations, transparency. Better compression than WebP but browser support still not strong.

Usability best practices: Images

Image loading techniques:

- Preloading
  - Load images before needed, so that they won't make the page appear slower than it actually is
- Lazy loading
  - Only load images when the page has scrolled to them, instead of loading everything when the page loads
- Progressive images
  - Have files of different qualities / sizes; load the smaller (worst quality) first, then load the improved one

Accessibility

Accessibility

Refers to making sure the website or app can be used comfortably by **all** possible users, including those with disabilities.

Understanding disabilities (video)

Accessibility

From this slideshow from w3.org:

People with disabilities should be able to

- Perform the same functions,
- Receive the same information,
- Participate as producers and consumers

without having to ask for assistance and without an special “text-only version”.

Accessibility

650 million people worldwide have some form of disability

- Includes 12.1% of US adults
- Includes 16% of working-age adults in the UK
- Over 30% of us will have some form of disability by the time we retire.

Photo by Marcus Aurelius: https://www.pexels.com/photo/woman-in-orange-tank-top-sitting-on-black-wheelchair-4064696/

Accessibility

Types of disability:

- Permanent: vision, hearing, physical, condition, learning…
- Temporary: broken arm…
- Situational: driving, environmental (glare, noise), being a new parent…

Photo by Marcus Aurelius:
https://www.pexels.com/photo/woman-in-orange-tank-top-sitting-on-black-wheelchair-4064696/

Accessibility: examples of cases to care about

Blind users: They may use screen readers to navigate their computers. May or may not read Braille.

- Images should have text descriptions (`alt` and `title` attributes).
- Page hierarchy should be the correct one - this may be rendered as voices in different volumes
- Navigation should be rendered using the right tags and attributes (such as `<nav>`)
- Can complement visual cues with audio cues

Photo by Eren Li: https://www.pexels.com/photo/photo-of-person-using-braille-7188797/

Accessibility: examples of cases to care about

Low vision users: May see text, icons, etc. but struggle with low contrast, differentiating between colors, or struggle with small text.

- Text should not be too small.
- Text should have enough contrast.
- Information should not be conveyed by color alone (eg. “go” = green, “stop” = red); text/icons should also be used
- Can complement visual cues with audio cues

This text will be difficult to read for some readers. It is smaller and it has poor contrast.

This will be even worse.

Did you even notice this was here?

Stop

Go

Accessibility: examples of cases to care about

Physical impairments: May not be able to use a mouse or keyboard well; may have reduced movement; may be sensitive to light changes

- Avoid sudden changes in light (eg. in animations)
- Audio cues may be useful too
- Inputs should be easy to use

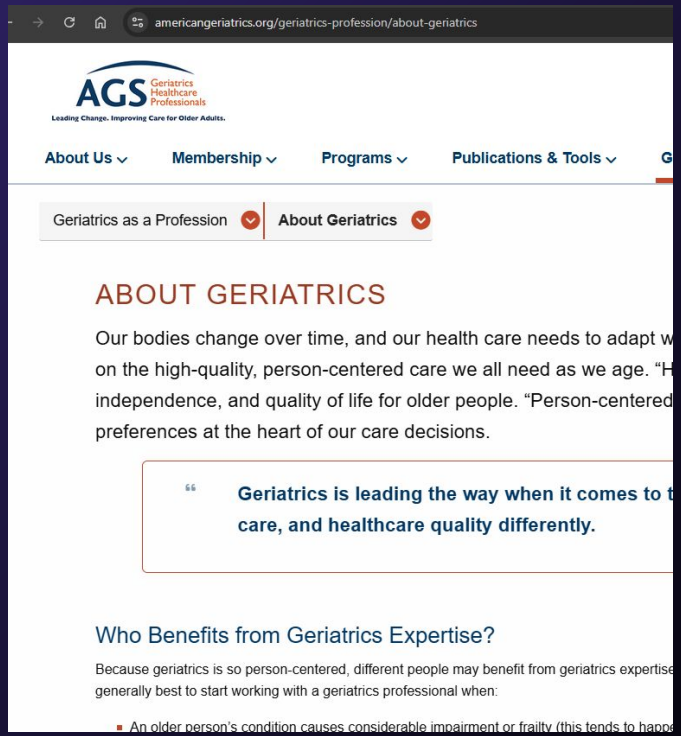
May use dictation software, foot switches, sip and puff devices, specialized keyboard and mice

Sip-and-puff device. Image from Wikipedia.

Accessibility: examples of cases to care about

Cognitive and intellectual disabilities: Covers: memory problems (eg. dementia), problem solving issues, math comprehension, attention issues, reading comprehension, etc.

- Content should have good contrast and size
- Avoid complex terms if possible
- Avoid unnecessarily long text
- Use more than one way to explain things; eg. text and illustration

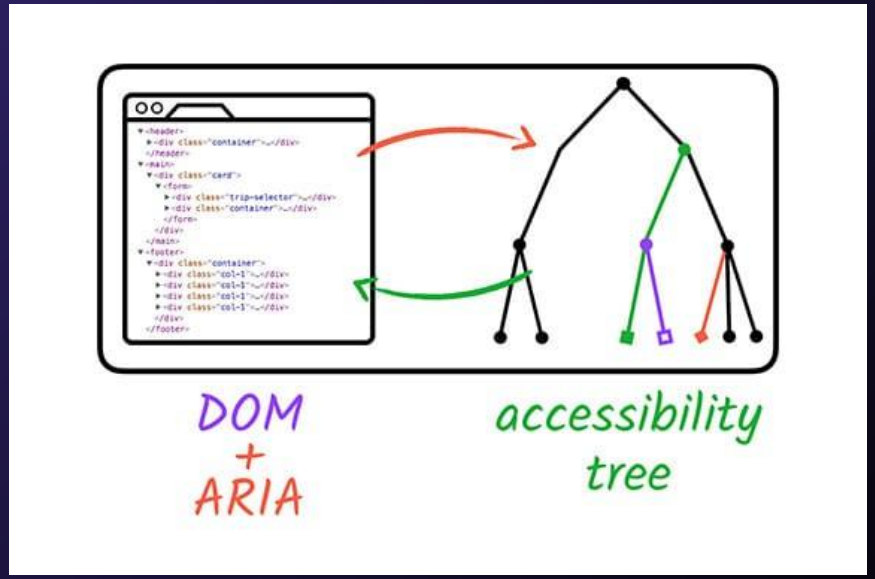


Screenshot from americangeriatrics.org, a website with good contrast and text size

WAI-ARIA conformance

The best way to make sure the website or app will be accessible is to conform to **WAI-ARIA standards**.

Using the right HTML tag covers many cases.



Source: [web.dev](web.dev)

WAI-ARIA HTML attributes

Some HTML tag attributes are specific to ARIA. They fulfill a specific mission to convey meaning. Here are a few ones.

WAI-ARIA HTML attributes

`aria-disabled`	Means an element can be perceived (it's there), but cannot be used. Note: **This won't block clicks, etc.** You need to do that through JS.
`aria-hidden="true"`	For decorative or redundant elements or text that we want a screen reader to skip. Example: emojis.

```html
<button aria-disabled="true" onclick="return false;">Submit</button>

<button>
    <span aligned="true">🔍</span>
    Search
</button>
```

WAI-ARIA HTML attributes

`aria-current`	Indicates which element is “active”, for example, the current page in a navigation bar, or the selected date in a calendar.

```
<!-- ──────────────────────────────────────────────────────────
  aria-current
  Indicates the current item within a set.
  Values: page | step | location | date | time | true
────────────────────────────────────────────────────────────── -->
<nav>
  <a href="/home">Home</a>
  <a href="/about" aria-current="page">About</a>
  <a href="/contact">Contact</a>
</nav>
```

WAI-ARIA HTML attributes

`aria-live`	Indicates elements, such as chat sections, that will periodically be updated. Tells screen readers to announce when there is a change.
`aria-busy`	For elements that are being loaded.

```
<!--
────────────────────────────────────────────────────────────────────────────────
  aria-live
  Tells screen readers to announce dynamic
  content changes automatically.
  Values: off | polite | assertive
────────────────────────────────────────────────────────────────────────────────
-->
```

```
<!--
────────────────────────────────────────────────────────────────────────────────
  aria-busy
  Signals that a region is still loading.
  Screen readers wait before reading its content.
────────────────────────────────────────────────────────────────────────────────
-->
<div role="feed" aria-busy="true" id="results">
```

WAI-ARIA HTML attributes

`aria-checked`, `aria-selected`, `aria-pressed`	States of a form control. Useful when non-standard components are used instead of `<input>` tags.

```html
<!-- ──────────────────────────────────────────────────────
  aria-selected
  Marks the chosen item in a selection widget
  (tabs, listboxes, grids, etc.).
────────────────────────────────────────────────────────── -->
<ul role="tablist">
  <li role="tab" aria-selected="true">Overview</li>
  <li role="tab" aria-selected="false">Details</li>
  <li role="tab" aria-selected="false">Reviews</li>
</ul>
```

WAI-ARIA HTML attributes

`aria-controls`	Identifies the controls (such as a “play” or “stop” button) of an interactive widget (such as a video player).

```
<!-- ──────────────────────────────────────────────────
  aria-controls
  Creates a programmatic link between a control
  and the element it affects (by ID).
────────────────────────────────────────────────────── -->
<button aria-controls="sidebar" onclick="toggleSidebar()">
  Toggle Sidebar
</button>
<aside id="sidebar">…</aside>
```

WAI-ARIA HTML attributes

`aria-haspopup`	Warns users that this element triggers a popup or dialog.

```
<!-- ──────────────────────────────────────────────────
  aria-haspopup
  Warns users that activating this element
  opens a popup (menu, listbox, dialog, etc.).
  Values: true | menu | listbox | tree | grid | dialog
────────────────────────────────────────────────────── -->
<button aria-haspopup="menu" aria-expanded="false" aria-controls="user-menu">
  Account ▾
</button>
<ul role="menu" id="user-menu" hidden>
  <li role="menuitem">Profile</li>
  <li role="menuitem">Sign Out</li>
</ul>
```

Accessibility best practices

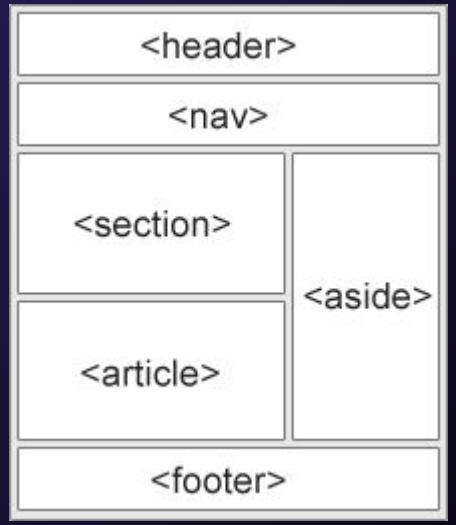
- Landmark regions
- Accessible names and descriptions
- Structural roles

Plus others that appear here.

Accessibility best practices: Landmark regions

Refers to using specific parts of the page when creating its structure, and making sure all “visible” content is inside this structure of “regions”.

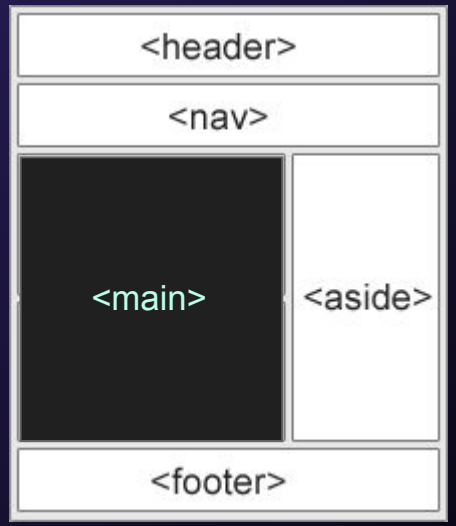
Remember this?



Accessibility best practices: Landmark regions

Refers to using specific parts of the page when creating its structure, and making sure all “visible” content is inside this structure of “regions”.

We’ll tweak it to this:

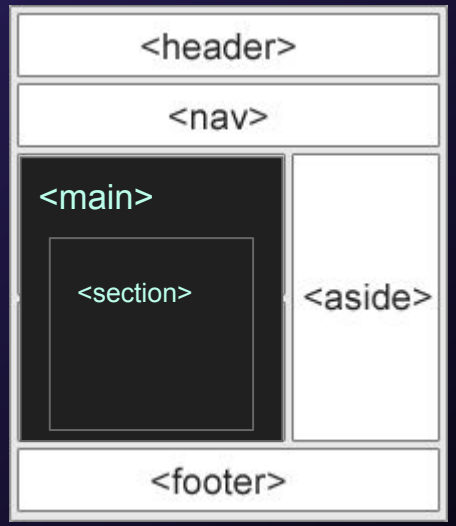


Accessibility best practices: Landmark regions

Refers to using specific parts of the page when creating its structure, and making sure all “visible” content is inside this structure of “regions”.

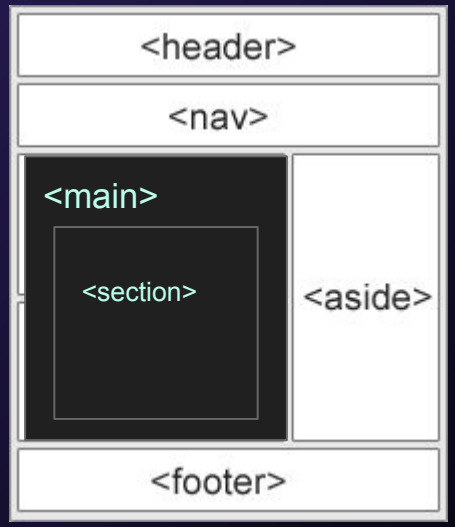
Also see: https://www.w3.org/WAI/ARIA/apg/practices/landmark-regions/

Define <section> as needed, and use accessible names:



Accessibility best practices: Landmark regions

```html
<!DOCTYPE html>
    <head></head>
    <body>
        <header></header>
        <nav></nav>
        <main>
            <section id="welcome" aria-labelledby="heading-welcome">
                <h1 id="heading-welcome">Welcome to this website!</h1>
                <p>Blah blah.</p>
            </section>
            <section id="intro" aria-label="Thoughts">
                Here are some of my thoughts.
            </section>
        </main>
        <aside></aside>
        <footer></footer>
    </body>
</html>
```



Accessibility best practices: Landmark regions

```html
<!DOCTYPE html>
    <head></head>
    <body>
        <header></header>
        <nav></nav>
        <main>
            <section id="welcome" aria-labelledby="heading-welcome">
                <h1 id="heading-welcome">Welcome to this website!</h1>
                <p>Blah blah.</p>
            </section>
            <section id="intro" aria-label="Thoughts">
                Here are some of my thoughts.
            </section>
        </main>
        <aside></aside>
        <footer></footer>
    </body>
</html>
```

All of these are “landmark regions” now.

Accessibility best practices: Landmark regions

```html
<!DOCTYPE html>
    <head></head>
    <body>
        <header></header>
        <nav></nav>
        <main>
            <section id="welcome" aria-labelledby="heading-welcome">
                <h1 id="heading-welcome">Welcome to this website!</h1>
                <p>Blah blah.</p>
            </section>
            <section id="intro" aria-label="Thoughts">
                Here are some of my thoughts.
            </section>
        </main>
        <aside></aside>
        <footer></footer>
    </body>
</html>
```

These five are “landmark regions” because they are special, and because they’re direct children of <body>

Accessibility best practices: Landmark regions

```html
<!DOCTYPE html>
    <head></head>
    <body>
        <header></header>
        <nav></nav>
        <main>
            <section id="welcome" aria-labelledby="heading-welcome">
                <h1 id="heading-welcome">Welcome to this website!</h1>
                <p>Blah blah.</p>
            </section>
            <section id="intro" aria-label="Thoughts">
                Here are some of my thoughts.
            </section>
        </main>
        <aside></aside>
        <footer></footer>
    </body>
</html>
```

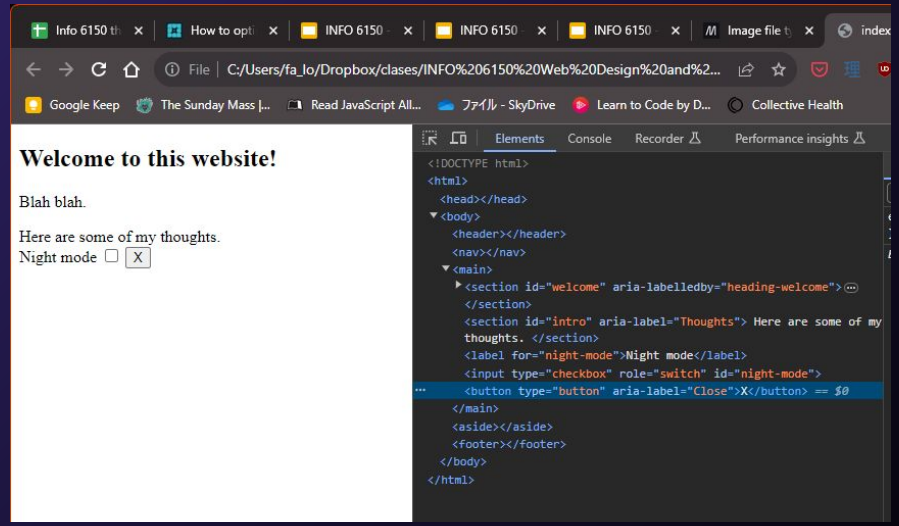
`<section>` with `aria-label` or `aria-labelled by` (“accessible names”) are also landmark regions. Names should be unique.

Accessibility best practices: Accessible names and descriptions

Every element should have a name that conveys what it is and distinguishes from other elements.

- The **visible** text inside the element can become its name.
- If no visible text, use aria-label:

```
<button type="button" aria-label="Close">X</button>
```

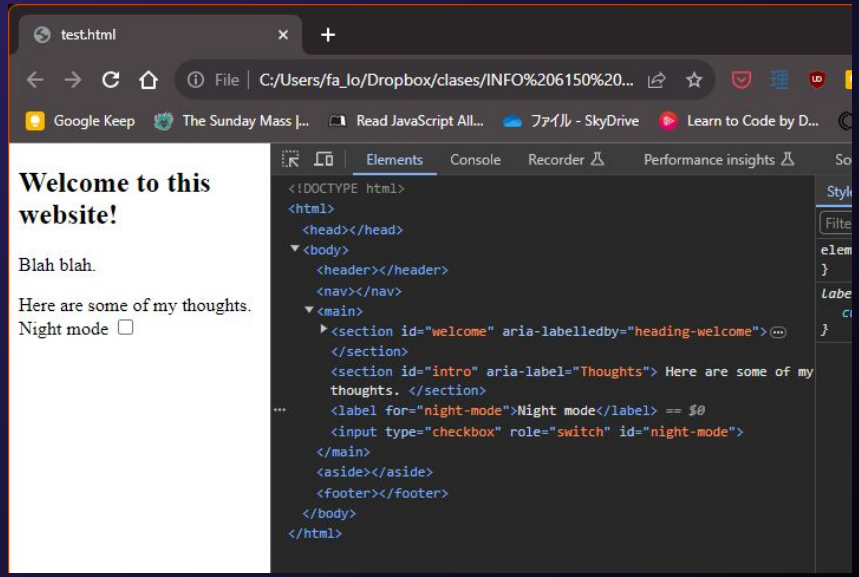


Accessibility best practices: Accessible names and descriptions

Every element should have a name that conveys what it is and distinguishes from other elements.

- The **visible** text inside the element can become its name.
- If no visible text, use aria-label
- If the name of an element is given by another element, use aria-labelledby:

```
<section id="welcome"
aria-labelledby="heading-welcome">
    <h1 id="heading-welcome">Welcome to this
website!</h1>
```

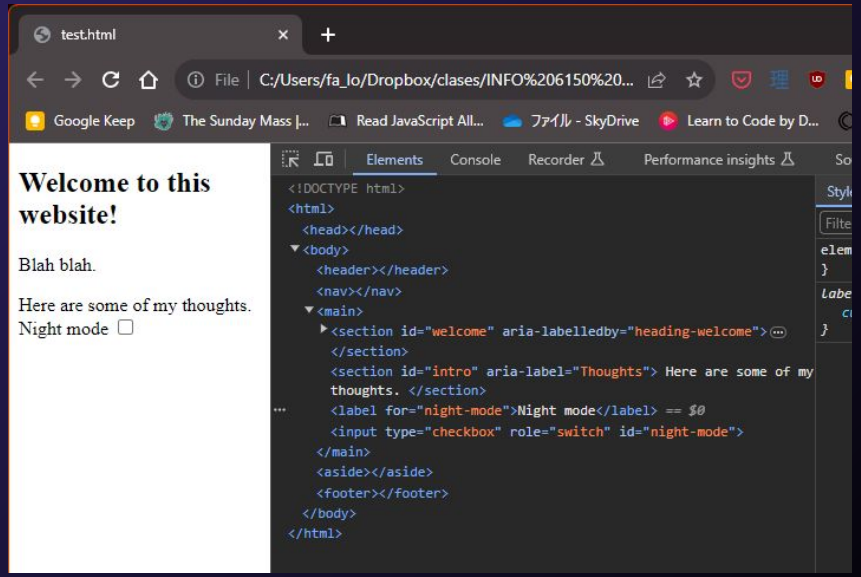


Accessibility best practices: Accessible names and descriptions

Every element should have a name that conveys what it is and distinguishes from other elements.

- The **visible** text inside the element can become its name.
- If no visible text, use aria-label
- If the name of an element is given by another element, use aria-labelledby
- For form inputs, use <label> instead.

```
<label for="night-mode">Night mode</label>
<input type="checkbox" role="switch"
id="night-mode">
```



More details here.

Accessibility best practices: Structural roles

80% of times this means “use the right HTML tag for the function”.

Only when one is not available, use the `aria-role` attribute.

Check the full list here if curious. A partial screenshot:

All Structural Roles and Their HTML Equivalents

The following table lists all structural roles defined in ARIA 1.2. As described above in the section on When to Use Structural Roles, use an equivalent HTML element instead of an ARIA structural role unless the ARIA role does not have an HTML equivalent or one of the four circumstances that makes using ARIA necessary exists.

ARIA structural roles

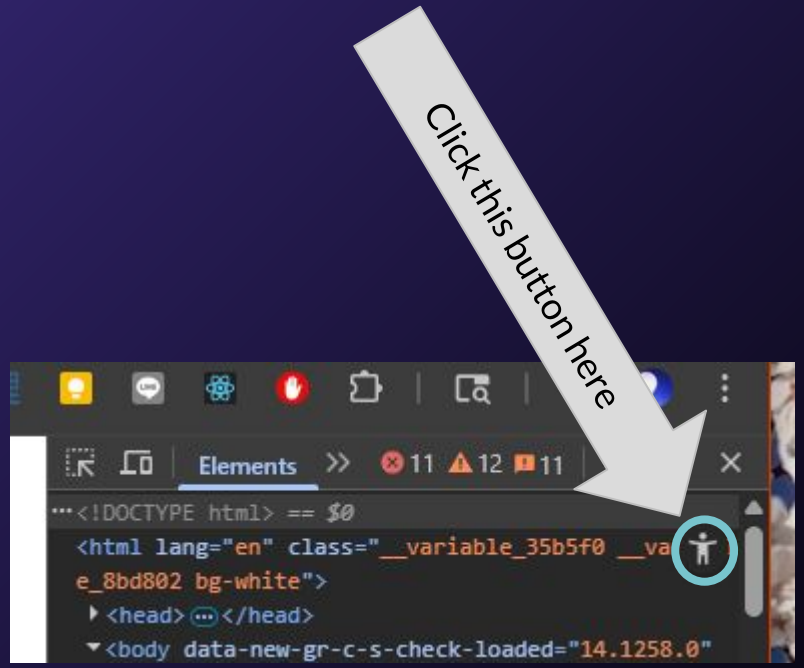
ARIA Role	HTML Equivalent
application	No equivalent element
article	article
blockquote	blockquote
caption	caption
cell	td
code	code
columnheader	th
definition	dd
deletion	del
directory	No equivalent element
document	No equivalent element
emphasis	em
feed	No equivalent element

The Accessibility Tree

This is similar to the DOM tree: the browser creates it when it loads a page. However, it is composed out of the ARIA roles of the elements of the page.

By looking at it, you can diagnose and fix accessibility problems, i. e., how a screen reader will interpret your page.

Look at it in the Elements tab of Developer Tools.



You Might Not Need Lodash

youmightnotneed.com/lodash

You Might Not Need **lodash** date-fns css

toggle dark mode

YOU MIGHT NOT NEED LODASH

But **you should use Lodash**.

It's a great library, well crafted, battle tested and with a very skilled and active community contributing. **The goal of this project is *NOT* to provide drop in replacements**, but to show how to achieve similar functionalities in plain Javascript, to understand how things work behind the hood.

Being a learning platform, **some implementations have been simplified to make them more digestible, and they might miss edge cases** covered in the original version.

If you think something important is missing, or plain wrong, feel free to open an issue, just be aware we are not aiming at feature parity. There are also quite a few methods yet to be ported; please help contributing on github.

Search...

Show tests?

Elements Console Recorder Sources Network Performance

```
RootWebArea "You Might Not Need Lodash" focusable: true
  url: https://youmightnotneed.com/lodash
  navigation ""
    link "You Might Not Need" focusable: true
      url: https://youmightnotneed.com/
    link "lodash" focusable: true
      url: https://youmightnotneed.com/lodash
    link "date-fns" focusable: true
      url: https://youmightnotneed.com/date-fns
    link "css" focusable: true
      url: https://youmightnotneed.com/css
    button "toggle dark mode" focusable: true
    link "" focusable: true
      url: https://github.com/cedmax/youmightnotneed
  heading "YOU MIGHT NOT NEED LODASH"
    StaticText "YOU MIGHT NOT NEED"
    strong ""
  paragraph ""
    StaticText "But "
    strong ""
    StaticText "."
  paragraph ""
    StaticText "It's a great library, well crafted, battle
    tested and with a very skilled and active community
    contributing. "
    strong ""
    StaticText ", but to show how to achieve similar
    functionalities in plain Javascript, to understand how
    things work behind the hood."
  paragraph ""
    StaticText "Being a learning platform, "
    strong ""
    StaticText " covered in the original version."
  paragraph ""
    StaticText "If you think something important is missing,
    or plain wrong, feel free to "
    link "open an issue" focusable: true
      url: https://github.com/cedmax/youmightnotneed/issues
```

Styles Computed Layout

Filter :hov .cls

```
element.style {
}

body {                                  <style>
  font-family: HelveticaNeue-Light,
    "Helvetica Neue Light",
    "Helvetica Neue", Helvetica,
    Arial, "Lucida Grande", sans-
    serif;
  font-weight: 300;
  font-size: 16px;
  max-width: 100%;
  margin: ▸ 0px;
  padding: ▸ 0px;
}

body {                  23a8648c9283e660.css:1
  ~~margin: ▸ 0 auto;~~
  ~~max-width: 750px;~~
  ~~padding: ▸ 2rem;~~
  border-radius: ▸ 6px;
  overflow-x: hidden;
  word-break: break-word;
  overflow-wrap: break-word;
  background: ▸ var(--nc-bg-1);
  color: var(--nc-tx-2);
  ~~font-size: 1.03rem;~~
  line-height: 1.5;
}

* {                      23a8648c9283e660.css:1
  ~~margin: ▸ 0;~~
  ~~padding: ▸ 0;~~
}

body {                  user agent stylesheet
  display: block;
  ~~margin: ▸ 8px;~~
}

Inherited from html

:root {                                 <style>
  --nc-tx-1: #e8e8e8;
  --nc-tx-2: #ccc;
  --nc-bg-1: #111111;
  --nc-bg-2: #181818;
  --nc-bg-3: #3a3a3a;
  --nc-lk-1: #99BFFF;
  --nc-lk-2: rgb(176, 188, 198);
```